djApp — Manuale d’uso

Alessandro Pezzali — PezzaliApp

17 aprile 2026

djApp — Manuale d’uso

Versione documento: 2.0 (riscritto su bundle verificato) **App:** djApp by PezzaliApp **URL produzione:** https://www.alessandropezzali.it/djapp-new/ **Stack:** React 18 + Vite 5 + Web Audio API + Zustand **Ambito:** Uso completo su macOS, Windows, Android, iPhone/iPad

---

Indice

1. Cos’è djApp
2. Requisiti
3. Installazione come PWA
4. Panoramica dell’interfaccia
5. Caricamento dei brani
6. Deck A e Deck B — comandi completi
7. Mixer centrale
8. Effetti — COLOR FX e BEAT FX
9. Scorciatoie da tastiera
10. Overlay aggiuntivi — Recorder WAV e Beatmatch
11. Workflow completo — mixare due brani
12. Uso su mobile
13. Auto-aggiornamento PWA
14. Risoluzione problemi
15. Note tecniche

---

1. Cos’è djApp

djApp è un’applicazione web per il mixaggio di due tracce audio in tempo reale, che gira direttamente nel browser senza plugin. È una **Progressive Web App (PWA)** installabile su macOS, Windows, Android e iOS, con interfaccia a due deck, mixer centrale con EQ 3 bande per canale, crossfader, catena di effetti (COLOR FX + BEAT FX), hot cue, loop engine, sync automatico con allineamento di fase, e analisi BPM automatica via worker dedicato.

Costruita sopra la Web Audio API nativa. Nessun account, nessun tracking, i brani restano sul dispositivo.

2. Requisiti

2.1 Browser supportati

Piattaforma	Browser consigliato	Note
macOS	Chrome (ultima versione)	Esperienza piena, install PWA nativa
macOS	Safari 17+	Funziona, install via "Aggiungi al Dock"
Windows	Chrome, Edge	Esperienza piena
Android	Chrome	Esperienza piena, PWA full-screen
iPhone/iPad	Safari	Obbligatorio (Chrome iOS = Safari sotto)

2.2 Connessione e dispositivo

- **Prima apertura**: serve internet (si scarica il bundle)
- **Uso successivo**: l'interfaccia funziona offline (Service Worker). I brani si caricano localmente
- **HTTPS obbligatorio** per l'AudioContext
- **Cuffie consigliate** per il monitoraggio

2.3 Limiti hardware browser

Su **iOS Safari** il `playbackRate` è limitato dal browser all'intervallo **[0.5, 4.0]**: scratch estremi oltre questi limiti vengono clampati automaticamente. Il resto delle piattaforme non ha questo limite.

---

3. Installazione come PWA

3.1 macOS — Chrome

1. Apri Chrome e vai su `https://www.alessandropezzali.it/djapp-new/`
2. Attendi il caricamento completo
3. Sul lato destro della barra indirizzi, icona **monitor con freccia ⊕** ("Installa djApp")
4. Click → **Installa**
5. L'app appare in `/Applicazioni/` e in Launchpad

Menu alternativo: tre puntini ⋮ → `Trasmetti, salva e condividi` → `Installa pagina come app...`

3.2 macOS — Safari

Menu **File** → **Aggiungi al Dock...** → scegli nome e icona → `Aggiungi`.

3.3 Windows — Chrome o Edge

Barra indirizzi, icona ⊕ → `Installa`. L'app appare in Start Menu.

3.4 Android — Chrome

Bottom-sheet automatico **"Installa app"** o menu ⋮ → **"Installa app"** → `Installa`.

3.5 iPhone / iPad — Safari

Icona **Condividi** (quadrato + freccia in alto) → scroll in basso → **"Aggiungi alla schermata Home"** → `Aggiungi`.

---

4. Panoramica dell'interfaccia

4.1 Layout desktop

Cinque zone dall'alto in basso:

1. **Header**: logo `djApp · by PezzaliApp` a sinistra, indicatore ● `READY` a destra (si accende quando l'AudioContext è pronto)
2. **Waveform Overview**: forme d'onda dei due deck affiancate (A blu, B verde), con zoom temporale indicato (es. `30s`, `120s`)
3. **Zona performance** (centrale): 3 colonne — Deck A a sinistra, Mixer al centro, Deck B a destra
4. **Barra effetti**: COLOR FX a sinistra, BEAT FX a destra
5. **Library**: campo di ricerca tracce, pulsante `+ IMPORT`, area drop-files

4.2 Layout mobile

Su schermi stretti appaiono **tab di navigazione** in basso (verificate nel bundle):

- ◉ `DECK A`
- ⇌ `MIXER`
- ◉ `DECK B`
- ♪ `LIBRARY`

Si tocca la tab per mostrare solo quella sezione.

---

5. Caricamento dei brani

5.1 Drag & drop

Trascina un file audio dal sistema direttamente sulla waveform del deck A o B. Qualsiasi formato decodificabile da `AudioContext.decodeAudioData` (MP3, WAV, M4A, AAC, FLAC, OGG dipendono dal browser).

5.2 Pulsante LOAD FILE

Pulsante **`LOAD FILE`** sotto ogni deck → file picker del sistema.

Flusso tecnico verificato: 1. Il file viene letto come `ArrayBuffer` 2. `decodeAudioData` produce un `AudioBuffer` 3. Il buffer viene assegnato al deck, reset di cursore e loop 4. Il titolo del brano è il filename **senza estensione** 5. L'artista è impostato a `"Local File"` 6. Dopo 500 ms parte l'analisi BPM in un Web Worker dedicato

5.3 Library + IMPORT

In basso nella zona LIBRARY: campo `Search tracks...`, pulsante **`+ IMPORT`** per aggiungere brani alla libreria interna. Ogni riga della libreria mostra titolo, artista, BPM e durata, e ha un pulsante **×** per rimuoverla.

Doppio click su un brano della library → carica sul deck selezionato (con pulsante **`FROM LIBRARY`** del deck).

5.4 Analisi BPM automatica

L'analisi BPM **è reale e funzionante**. Algoritmo implementato:

- Downsample del segnale audio per fattore 10
- Calcolo dell'energia RMS a finestre brevi
- **Autocorrelazione** tra offset corrispondenti a 60–200 BPM
- Il lag con correlazione massima determina il BPM
- Normalizzazione in intervallo 80–160 BPM (moltiplica o divide per 2 se fuori range)
- Arrotondamento a 1 decimale

Il BPM rilevato viene usato per: display, auto-loop, sync automatico, jump a beat.

---

6. Deck A e Deck B — comandi completi

Ogni deck ha la seguente catena di controlli, verificata nel bundle.

6.1 Track info e tempo

In alto: titolo del brano, artista, tempo corrente e **tempo rimanente** in formato `m:ss`, **BPM** con una decimale.

6.2 Jog wheel — NUDGE / SCRATCH

Il grande disco circolare è il jog wheel, supportato da **mouse + touch multitouch** (due dita su iPhone su due jog diversi funzionano in parallelo grazie a `useMultitouch` che mappa ciascun touch al proprio deck tramite `elementFromPoint`).

Sotto il jog c'è un **pulsante toggle** che alterna tra due modalità:

- ⟳ **`NUDGE`** (default): il jog fa pitch-bend temporaneo. Ruota in senso orario → accelerazione momentanea. Ruota antiorario → rallentamento momentaneo. Rilascio → torna al pitch impostato
- ⦿ **`SCRATCH`**: il jog diventa una superficie scratch — la riproduzione segue direttamente la posizione del gesto (implementato via manipolazione `playbackRate`, non AudioWorklet)

6.3 KEY shift

Sotto il jog-mode toggle: `KEY ♭ 0 ♯ M.TEMPO`

Verificato implementato:

- ♭ **(bemolle)**: abbassa la tonalità di 1 semitono (fino a **-6 semitoni**)
- **`0`**: tonalità originale (reset — valore corrente visibile)
- ♯ **(diesis)**: alza di 1 semitono (fino a **+6 semitoni**)
- **`M.TEMPO`**: toggle Master Tempo. Quando attivo, le variazioni di KEY non alterano il tempo del brano; quando disattivo, pitch e tonalità sono accoppiate

6.4 SYNC — implementato

Pulsante ⇌ **`SYNC`**. Cablato a un handler reale:

- Quando attivato: imposta l'altro deck come **master**, calcola il rapporto `BPM_master / BPM_this`, applica il `playbackRate` risultante
- **In più esegue l'allineamento di fase**: confronta le posizioni dei due deck modulo la durata di un beat, calcola il delta di fase, lo corregge con un micro-nudge
- Lo stato del pulsante diventa evidenziato quando sync è attivo
- Secondo click: `unsyncDeck` — disattiva il sync per questo deck

6.5 QUANTIZE

Pulsante ◈ **`QUANTIZE`**. Cablato al flag `quantize` nello store. Quando attivo, forza gli eventi ritmici a scattare sul beat grid invece del momento esatto del click.

6.6 PITCH slider

Slider orizzontale con percentuale a destra.

Verificato nel bundle:

- **Range**: `playbackRate` da `0.7` a `1.3`
- **Step**: `0.001`
- **Visualizzazione**: `±X.X%` dove `X = (playbackRate - 1) × 100`; quando a 1.0 esatto mostra `±0%`
- Range percentuale effettivo: **da -30% a +30%**

6.7 Trasporto — CUE / PLAY / STOP

Tre pulsanti sotto il PITCH:

- **`CUE`**: funziona **a pressione**
  - `mousedown` / `touchstart` → va al cue point e riproduce
  - `mouseup` / `touchend` → torna in pausa
  - Colore evidenziato quando `cuePoint > 0`
- ▶ / ⏸: toggle play/pause
- ■: stop (seek a 0, pausa)

6.8 HOT CUE 1-8 + banchi A/B/C/D

Griglia `1 2 3 4 / 5 6 7 8` con header `HOT CUE` + `CLR` e selettore banchi `A B C D`.

Verificato implementato:

- **Click su uno slot vuoto**: memorizza la posizione corrente come cue point. Range valido: posizione tra 0 e durata del brano
- **Click su uno slot con cue memorizzato**: seek alla posizione, avvia play se non già in play
- **Banchi A/B/C/D**: quattro set da 8 cue = **32 hot cue totali per deck**

Il pulsante `CLR` seguito da un numero cancella quel hot cue (UI presente, handler nel bundle).

6.9 LOOP engine — implementato

Verificato nel bundle come `class Ue` (LoopEngine). Ha questi metodi reali:

- **Barra LOOP** con `1/4 | 1/2 | 1 | 2 | 4 | 8 | 16`: click chiama `autoLoop(beats)` che calcola la durata in secondi dal BPM e crea un loop dalla posizione corrente
- **`IN`**: chiama `setLoopIn()` — memorizza la posizione come inizio loop (o resetta se premuto due volte)
- **`OUT`**: chiama `setLoopOut()` — imposta la fine del loop e lo attiva; se OUT è prima di IN, corregge automaticamente
- **`LOOP`**: chiama `toggleLoop()` — enter/exit del loop corrente
- ◀◀: chiama `halve()` — dimezza la durata del loop attivo
- ▶▶: chiama `double()` — raddoppia la durata

Il loop usa un `requestAnimationFrame` continuo per controllare quando `position >= loopOut` e fare seek a `loopIn` (`_startLoopCheck`).

6.10 JUMP

Barra con pulsanti per saltare di battute intere. Cablati via **`beatJump(n)`** del LoopEngine (da tastiera: ← / → / ↓ / ↑ saltano di 1 beat — vedi sezione shortcut).

6.11 LOAD FILE / FROM LIBRARY

Due pulsanti in basso nel deck. `LOAD FILE` apre il file picker. `FROM LIBRARY` carica il brano attualmente selezionato nella library.

---

7. Mixer centrale

7.1 Display BPM e Δ

In alto: i due BPM effettivi e la loro differenza. Con il patch overlay Beatmatch il display segue il pitch corrente (vedi sezione 10).

7.2 PHASE meter

Indica lo sfasamento tra i beat dei due deck. Pallino centrato = in fase. Il testo ✓ **`IN SYNC`** appare quando il delta è sotto 0.1; altrimenti viene mostrato `Δ <valore>` in arancione.

7.3 Canali A e B — fader verticali

Ogni canale ha 4 fader impilati dall'alto in basso:

- **`HI`** — EQ banda alta (BiquadFilter `highshelf`, frequenza 4 kHz)
- **`MID`** — EQ banda media (BiquadFilter `peaking`, frequenza 1 kHz, Q=1)
- **`LO`** — EQ banda bassa (BiquadFilter `lowshelf`, frequenza 200 Hz)
- **`GAIN`** — volume del canale dopo gli EQ

Verificato nel bundle:

- **Range EQ**: gain da **-12 a +12 dB**, step 0.5
- **Range GAIN**: da 0 a 1, step 0.01
- **Valore iniziale gain**: 0.8

7.4 Crossfader A ↔ B

Slider orizzontale in basso, range 0–1 step 0.01. Etichette `A` e `B` ai lati, `CENTER` al centro.

---

8. Effetti — COLOR FX e BEAT FX

8.1 COLOR FX (4 slot sempre attivi)

Lista verificata nel bundle (array `Kn`):

ID	Label	Colore
`reverb`	**REVERB**	viola
`delay`	**DELAY**	arancione
`filter`	**FILTER**	blu
`flanger`	**FLANGER**	ciano

Ogni slot ha uno slider 0–100% che controlla il wet. Handler: `b.fx.setWet(id, value)`. I quattro effetti sono sempre "in linea", si attivano alzando lo slider.

8.2 BEAT FX (9 effetti, uno selezionabile)

Lista verificata (array `_e`):

ID	Label
`delay`	DELAY
`echo`	ECHO
`pingpong`	PING PONG
`reverb`	REVERB
`filter`	FILTER
`flanger`	FLANGER
`phaser`	PHASER
`roll`	ROLL
`trans`	TRANS

Divisioni ritmiche (array `In`):

Label	Valore
`1/8`	0.125
`1/4`	0.25
`1/2`	0.5
`1`	1.0
`2`	2.0
`4`	4.0

Valore di default: `1` (1 beat).

Controlli: - Slider wet 0-100% (default 70%) - Pulsante **`ON`**: enable/disable

Handler reali: `b.beatFX.setEffect(id, wet)`, `b.beatFX.setBeatDiv(value)`, `b.beatFX.setWet(wet)`, `b.beatFX.off()`.

---

9. Scorciatoie da tastiera

Estratte dal codice del bundle (funzione `ts`). Le shortcut sono ignorate se il focus è su `<input>` o `<textarea>`.

9.1 Globali

Tasto	Azione
`?`	Mostra/nascondi pannello help
`Escape`	Stop entrambi i deck

9.2 Deck A

Tasto	Azione
`Spazio`	Play / Pause
`Z`	Cue
`←`	Jump back 1 battuta
`→`	Jump forward 1 battuta
`Q`	KEY down (abbassa tonalità, min -6)
`E`	KEY up (alza tonalità, max +6)
`1 – 8`	Hot cue 1-8 (senza Cmd/Ctrl/Alt)

9.3 Deck B

Tasto	Azione
`Enter`	Play / Pause
`M`	Cue
`↓`	Jump back 1 battuta
`↑`	Jump forward 1 battuta
`O`	KEY down
`P`	KEY up
`F1 – F8`	Hot cue 1-8

10. Overlay aggiuntivi — Recorder WAV e Beatmatch

Due overlay JavaScript vanilla sono caricati **prima del bundle React** e aggiungono funzionalità.

10.1 Recorder WAV (`recorder.js`)

Pill nero con bordo giallo `#e8ff47` in **basso a destra**.

Funzionamento: intercetta `AudioContext.prototype.constructor` e `AudioNode.prototype.connect`, inserisce un tap `GainNode` sul master, da lì cattura via AudioWorklet i sample stereo Float32 che vengono codificati in **WAV PCM 16-bit stereo** alla sample rate dell'AudioContext (tipicamente 48 kHz su Chrome, 44.1 kHz su Safari).

Uso: click ● `REC` → mixa → click ■ `STOP` → download automatico del file `djapp-mix-YYYY-MM-DDTHH-MM-SS.wav`.

Limiti: - Solo audio del browser (non microfono o sistema) - Memoria: set > 60 min può saturare la RAM del browser - Registra **tutto il master**: deck, EQ, effetti, crossfader

10.2 Beatmatch (`beatmatch.js`)

Pannello draggable con bordo blu.

Funzione 1 — **Display BPM effettivo**: aggiorna i numeri `90.0 / 92.2 / Δ` nel Mixer centrale in base al `playbackRate` corrente di ogni deck (il display nativo mostra il BPM originale analizzato, non quello effettivo).

Funzione 2 — Pulsanti `SYNC B→A` e `SYNC A→B`: allineano il tempo bypassando lo slider PITCH visivo.

> ⚠️ **Nota importante**: il pulsante `⇌ SYNC` integrato nel deck è **più potente** dell'overlay Beatmatch perché allinea anche la **fase** (non solo il tempo). Per un sync completo usa il SYNC integrato. L'overlay Beatmatch è utile quando vuoi vedere i BPM effettivi in tempo reale mentre manovri il pitch manualmente.

UI del pannello: - Header trascinabile (click + drag) - Freccia `▾ / ▸` per collassare - Posizione e stato collapsed salvati in `localStorage`

---

11. Workflow completo — mixare due brani

Procedura verificata su bundle reale, usando le feature che **funzionano davvero**.

11.1 Preparazione

1. Carica **Brano A** sul deck A (drag&drop o `LOAD FILE`)
2. Attendi l'analisi BPM (~2-5 secondi)
3. Carica **Brano B** sul deck B
4. Crossfader tutto a sinistra (posizione `A`)
5. Canale A volume 100%, canale B volume 0
6. EQ di entrambi i canali al centro (0 dB)

11.2 Beatmatching automatico (consigliato)

1. PLAY su deck A
2. PLAY su deck B (con volume a 0, si sente solo A)
3. Click **`⇌ SYNC`** sul deck B → il deck B viene allineato **in tempo E fase** al deck A automaticamente

Il pulsante SYNC diventa evidenziato. L'indicatore PHASE deve mostrare **`✓ IN SYNC`**.

11.3 Beatmatching manuale (alternativa)

1. Sposta il PITCH del deck B finché il BPM effettivo coincide con quello di A (uso il pannello Beatmatch overlay per vedere il BPM effettivo)
2. Allinea la fase:
   - Premi `M` (cue deck B) per riposizionare
   - Al beat 1 del deck A, premi `Enter` per far partire B
   - Osserva il PHASE meter
   - Correggi con il jog (modalità NUDGE): senso orario se B in ritardo, antiorario se in anticipo

11.4 Transizione con EQ swap

Circa 16–32 battute:

1. **Battute 1-4**: abbassa gradualmente `LO` del canale A (toglie i bassi di A), alza `LO` del canale B fino al centro (inserisce i bassi di B). I kick non si pestano più
2. **Battute 5-12**: alza gradualmente il volume GAIN del canale B fino al 100%
3. **Battute 13-20**: muovi il crossfader da A verso B
4. **Battute 21-32**: crossfader tutto su B, volume canale A a 0, EQ A resettati

11.5 Registrazione

Usa l'overlay Recorder:

1. Prima di iniziare la sessione, click ● `REC`
2. Mixa normalmente
3. Alla fine click ■ `STOP` → scarica il `.wav`

---

12. Uso su mobile

12.1 Touch

Il jog wheel è **multitouch reale**. Su iPhone e iPad due dita possono manovrare contemporaneamente due jog diversi. Ogni touch è mappato al proprio deck tramite `elementFromPoint` al momento di `touchstart` e rimane legato a quel deck per tutta la gesture.

12.2 Tabs mobile

Su schermi stretti, sotto la barra effetti appaiono 4 tab: ◉ `DECK A`, ⇌ `MIXER`, ◉ `DECK B`, ♪ `LIBRARY`. Si tocca per cambiare sezione.

12.3 Limiti iOS

- `playbackRate` limitato a `[0.5, 4.0]` dal browser — scratch estremi vengono clampati
- Keep-awake: tieni lo schermo acceso durante un set
- Notifiche / chiamate interrompono l'AudioContext — attiva "Non disturbare"

12.4 Limiti Android

- Cuffie Bluetooth introducono latenza (100–300 ms): per beatmatching usa cuffie cablate USB-C o jack
- Chrome Android supporta nativamente l'installazione PWA

---

13. Auto-aggiornamento PWA

Verificato nel bundle: l'app registra il Service Worker, controlla aggiornamenti ogni **30 minuti**, e quando ne rileva uno mostra un **banner flottante centrato in basso**:

```
◉  Nuova versione disponibile   [AGGIORNA]
```

Click su `AGGIORNA` → il SW invia `SKIP_WAITING` al worker in attesa, la pagina si ricarica automaticamente, versione aggiornata.

Se non vedi il banner ma vuoi forzare l'aggiornamento: chiudi tutte le tab di djApp, attendi 1 minuto, riapri.

---

14. Risoluzione problemi

14.1 Chrome non offre l'install PWA

DevTools → `Application` → `Manifest`: controlla che `start_url` e `scope` siano `/djapp-new/`. Chrome elenca i problemi in fondo al pannello.

14.2 Audio assente o distorto

- Verifica ● `READY` verde nell'header
- Controlla volume sistema
- GAIN canale: se troppo alto con brani masterizzati forte, abbassa a ~0.6

14.3 BPM rilevato errato (es. metà o doppio)

L'analyzer normalizza in 80–160 BPM. Per brani fuori range (es. drum'n'bass a 170 BPM rilevato come 85, o reggaeton a 90 rilevato come 180), il display è sbagliato.

Workaround: usa pitch manuale + orecchio, oppure carica e ricontrolla.

14.4 SYNC non funziona

Serve che **entrambi i deck abbiano un BPM rilevato > 0**. Se un brano non ha BPM (analisi fallita), il SYNC viene ignorato. Controllare che il display mostri un numero, non `—.—`.

14.5 Service Worker blocca aggiornamenti

DevTools → `Application` → `Service Workers` → `Unregister`. Poi `Application` → `Storage` → `Clear site data`. Ricarica.

14.6 Shortcut tastiera non rispondono

Verifica che il focus non sia su un campo di input/textarea. Le shortcut vengono ignorate in quel caso.

---

15. Note tecniche

15.1 Stack verificato

- **React 18.3.1** + **Vite 5.4** + **Zustand 4.5.2** + Web Audio API nativa
- Dipendenze dev: `@vitejs/plugin-react`
- Bundle diviso via `manualChunks`: `react-core`, `zustand`, `index` (app)
- Base path build: `/djapp/` (il sito è servito da `/djapp-new/`, il manifest è stato corretto per allineare `start_url` e `scope`)

15.2 Audio engine

- **AudioEngine** (singleton): AudioContext, master chain, routing globale
- **DeckEngine** (per deck): buffer source, 3× BiquadFilter (LO/MID/HI), channelGain
- **LoopEngine** (per deck): `loopIn`, `loopOut`, `beatLength` (60/BPM), `autoLoop`, `halve`, `double`, `beatJump`
- **SyncEngine**: master/slave, phase alignment via micro-nudge
- **FX**: ColorFX (4 slot sempre attivi) + BeatFX (9 effetti, 1 alla volta)
- **BPM detector**: Web Worker con autocorrelazione su energia RMS

15.3 Unlock AudioContext

L'AudioContext viene inizializzato al primo `pointerdown` sull'app (requisito autoplay policy di tutti i browser).

15.4 Overlay esterni

I due overlay `recorder.js` e `beatmatch.js` sono JavaScript vanilla puro (no dipendenze), caricati da `<script src>` **prima del bundle React** in `index.html`. Funzionano intercettando `AudioContext.prototype` via class extension (recorder) o prototype chain (beatmatch).

15.5 Privacy

- Nessun tracking, nessun account
- I brani restano sul dispositivo, non vengono mai caricati su server
- Service Worker cacha solo l'interfaccia (shell + asset statici)

15.6 Licenza

Progetto personale di Alessandro Pezzali / PezzaliApp. Codice sul repository GitHub `pezzaliapp/djapp-new`.

15.7 Supporto e feedback

- Issue tracker: `https://github.com/pezzaliapp/djapp-new/issues`
- Sito autore: `https://alessandropezzali.it`

Fine del manuale.

Documento verificato sul bundle `index-N51-MIQm.js` del commit corrente. Aggiornato al 17 aprile 2026.